

Relatório Final - Sistema de Reserva de Salas de Estudo

1. Objetivo

Implementar e validar um sistema de reservas de salas com regras de cadastro de alunos, consulta de salas disponíveis, realização de reservas, controle de conflitos de horário e histórico de reservas.

2. Requisitos do sistema

Requisitos Funcionais (RF)

RF01: O sistema deve permitir o cadastro de um novo aluno.

RF02: O sistema deve permitir consultar as salas disponíveis.

RF03: O aluno deve poder realizar uma reserva de sala.

RF04: O sistema deve impedir reservas para horários já ocupados.

RF05: O sistema deve permitir consultar o histórico de reservas realizadas.

Requisitos Não Funcionais (RNF)

RNF01: O sistema deve ser implementado em Python.

RNF02: O código deve ser modularizado, com separação entre lógica e testes.

RNF03: O tempo de resposta das funções deve ser inferior a 3 segundos.

3. Estrutura do projeto

``src/`` — código-fonte do sistema

- ``src/models.py``

- ``src/exceptions.py``

- ``src/sistema_reservas.py``

``tests/`` — testes automatizados em pytest

- ``tests/test_sistema_reservas.py``

``README.md`` — instruções de uso

`requirements.txt` — dependências necessárias

`relatorio\_final.md` / `relatorio\_final.pdf` — relatório final

4. Plano de Testes Funcionais

O plano abaixo reúne os casos de teste funcionais do sistema, com os dados de entrada, os resultados esperados e os resultados obtidos.

Identificador	Objetivo do teste	Entrada	Resultado Esperado	Resultado Obtido (simulado ou real)	Status
TC01	Cadastrar um novo aluno com sucesso	Matrícula nova, nome e e-mail válidos	O aluno é cadastrado e aparece no sistema	Aluno cadastrado com sucesso	Aprovado
TC02	Impedir cadastro duplicado	Mesma matrícula em um segundo cadastro	O sistema rejeita o cadastro e informa erro	Erro lançado corretamente	Aprovado
TC03	Consultar salas disponíveis sem reservas	Data e horário livre	Todas as salas aparecem como disponíveis	Lista de salas retornada corretamente	Aprovado
TC04	Impedir reserva em horário já ocupado	Reserva em um horário previamente reservado	A reserva deve ser rejeitada e a sala não deve ficar disponível	Reserva bloqueada corretamente	Aprovado
TC05	Realizar uma reserva válida	Aluno cadastrado, sala livre e horário disponível	A reserva é criada com sucesso	Reserva criada corretamente	Aprovado

5. Testes Automatizados

O arquivo de testes é `tests/test\_sistema\_reservas.py`. Ele contém os seguintes casos automatizados:

```
`test_cadastrar_aluno_com_sucesso`  
  
`test_cadastrar_aluno_duplicado_lanca_erro`  
  
`test_consultar_salas_disponiveis_sem_reservas`  
  
`test_sala_reservada_nao_aparece_como_disponivel`  
  
`test_realizar_reserva_com_sucesso`  
  
`test_reserva_horario_ocupado_lanca_erro`  
  
`test_consultar_historico_de_reservas`  
  
`test_reserva_aluno_nao_cadastrado_lanca_erro`  
  
`test_reserva_sala_inexistente_lanca_erro`  
  
`test_tempo_resposta_reserva_abaixo_de_3_segundos`  
  
`test_consultar_historico_aluno_nao_cadastrado_lanca_erro`
```

Cada teste está comentado para explicar o cenário e o resultado esperado.

6. Validação executada

Os testes foram executados localmente com pytest.

Resultado dos testes

Total de testes: 11

Status: 11 passed

Tempo de execução: 0.83s

7. Evidências de testes

Comando de execução:

- `python -m pytest -q`
- `python -m pytest --cov=src --cov-report=term-missing`

Saída exibida no terminal confirma os 10 testes aprovados.

A cobertura foi gerada para o pacote `src`.

Trecho de código de um teste automatizado:

```
def test_realizar_reserva_com_sucesso(sistema):  
    """TC05: aluno cadastrado consegue reservar uma sala livre."""  
    sistema.cadastrar_aluno("2025001", "Vívian Silva", "vivian@email.com")  
    reserva = sistema.realizar_reserva("2025001", "S01", date(2026, 7, 1), time(14, 0))  
    assert reserva.codigo_sala == "S01"  
    assert reserva.matricula_aluno == "2025001"  
    assert reserva.id_reserva == 1
```

8. Cobertura de testes e análise

Cobertura total do pacote `src`: **100%**

Arquivos analisados:

- `src/\_\_init\_\_.py`: 100%
- `src/exceptions.py`: 100%
- `src/models.py`: 100%
- `src/sistema\_reservas.py`: 100%

Todas as linhas de código estão cobertas pelos testes.

O que está coberto

Cadastro de novo aluno

Rejeição de matrícula duplicada

Consulta de salas disponíveis antes e depois de reservas

Reserva de sala livre

Bloqueio de reservas em horários ocupados

Histórico de reservas para aluno cadastrado

Erro ao consultar histórico com aluno não cadastrado

Erro ao reservar com aluno não cadastrado

Erro ao reservar sala inexistente

Tempo de resposta abaixo de 3 segundos

Validação de matrícula inválida em consulta de histórico

O que não está coberto

Nenhuma linha de código deixou de ser coberta pelos testes — cobertura 100%.

9. Conclusão

A implementação atende completamente aos requisitos funcionais e não funcionais do sistema de reserva de salas. Os testes automatizados cobrem todos os cenários de uso, incluindo cenários de sucesso e tratamento de erros. A análise de cobertura mostra 100% de cobertura do código-fonte, garantindo que todas as funcionalidades foram validadas.

O teste adicional para consulta de histórico com matrícula inválida reforça a segurança do sistema e garante que erros de entrada sejam sempre tratados corretamente.